TRƯỜNG ĐẠI HỌC ĐỒNG THÁP
Khoa Công nghệ và Kỹ thuật

`IN4529 · CHƯƠNG 1`

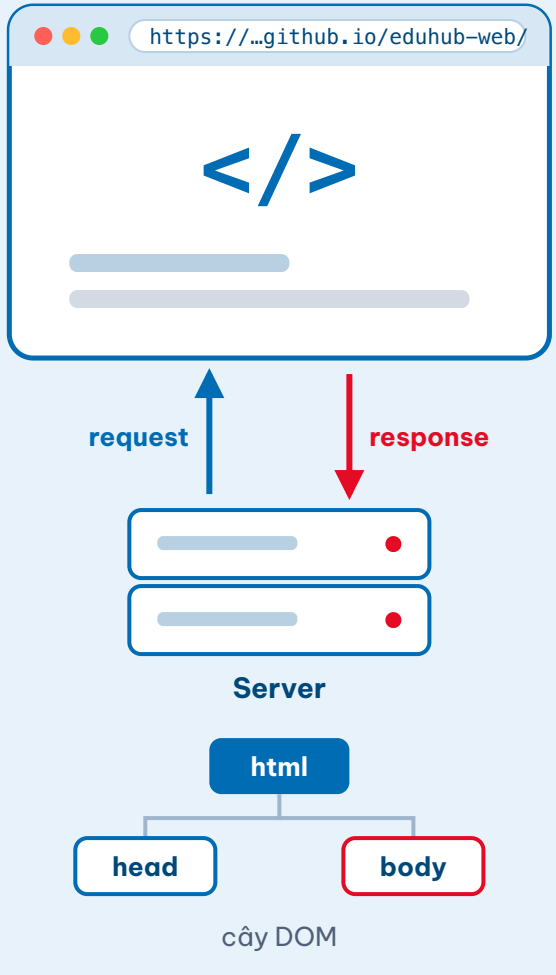
Web hoạt động thế nào & HTML

Lập trình Front End

Giảng viên **ThS. Nguyễn Tuấn Kiệt**
Năm học 2026–2027

`HTTP` `HTML5` `DevTools` `Git` `GitHub Pages`

0836 980 284 nguyentuankiet@dthu.edu.vn



Nội dung Chương 1

0 Giới thiệu môn IN4529

1.1 Gõ URL rồi chuyện gì xảy ra?

1.2 Frontend, Backend, Full-stack

1.3 Môi trường: DevTools, VS Code, Node.js, terminal

1.4 Git và GitHub cơ bản, GitHub Pages

1.5 HTML5: semantic, link, ảnh, bảng, form

1.6 Accessibility và SEO cơ bản

BT Bài tập, tổng kết, checkpoint

6 tiết lý thuyết · 6 tiết thực hành. Cuối chương: **EduHub Web** có 4 trang HTML chạy thật trên Internet bằng GitHub Pages.

Khởi động: bạn đã biết gì?

Chưa cần đúng — chỉ cần đoán. Cuối buổi ta quay lại 3 câu này.

CÂU 1 Gõ `dthu.edu.vn` rồi Enter: trang web đó đang **nằm ở đâu**, và nó đến máy bạn bằng cách nào?

CÂU 2 Chuột phải trên một trang web → *Xem nguồn trang* (View source). Bạn nghĩ đống chữ lạ đó là gì?

CÂU 3 Một trang web giống một ngôi nhà. Phần nào là **khung**, phần nào là **sơn**, phần nào là **điện**?



PHẦN 1 / 8

PHẦN 1

Giới thiệu môn IN4529

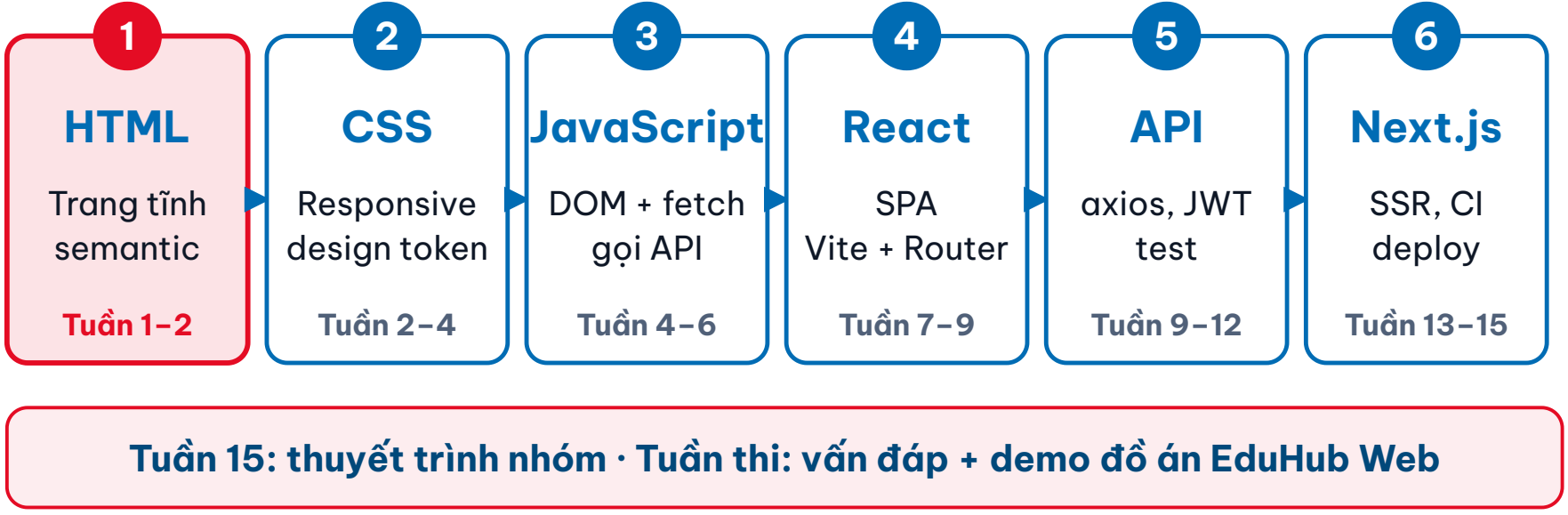
- Học xong môn này bạn làm được gì
- Lộ trình 6 chương và sản phẩm EduHub Web
- Cách đánh giá

Học xong môn này bạn làm được gì?

Chuẩn đầu ra (**CLO**) = những việc bạn làm được sau 15 tuần:

- **CLO1** — Giải thích web hoạt động thế nào; dựng giao diện **semantic, responsive, ai cũng dùng được** bằng HTML5, CSS hiện đại, Tailwind CSS
- **CLO2** — Lập trình tương tác bằng JavaScript/TypeScript; xây ứng dụng **React**, gọi REST API có đăng nhập (JWT) và có kiểm thử tự động
- **CLO3** — Xây và đưa lên mạng ứng dụng **Next.js** có SEO, test e2e, CI/CD
- **CLO4** — Làm việc nhóm qua Git/GitHub, tự học từ tài liệu chính thức, trình bày kết quả

Lộ trình 6 chương — mỗi chương xây thêm một tầng



Hình 1.1. Lộ trình 6 chương của học phần IN4529 (tự vẽ).

Chương sau dùng lại sản phẩm chương trước: HTML hôm nay sẽ được CSS ở Ch2 làm đẹp, JS ở Ch3 làm "sống".

Sản phẩm xuyên suốt: EduHub Web

Cổng web **câu lạc bộ (CLB) & sự kiện** cho sinh viên ĐH Đồng Tháp — 3 loại người dùng:

Khách

Xem CLB, sự kiện
Tìm kiếm, lọc
Xem chi tiết

Thành viên

Đăng ký, đăng nhập
Đăng ký tham gia sự kiện
Xem mã check-in

Quản trị

Thêm/sửa/xoá CLB, sự kiện
Xác nhận đăng ký
Check-in người tham gia

Hình 1.2. Ba vai trò người dùng của EduHub Web (tự vẽ).

Ch1 bắt đầu bằng phần **Khách**: 4 trang HTML tĩnh. Phần backend đã có sẵn (mock API, và EduHub .NET của môn IN4211).

Cách đánh giá



THÀNH PHẦN	NỘI DUNG	TRỌNG SỐ
Chuyên cần	Có mặt, đúng giờ, làm "Thử ngay" trên lớp	**0.1**
Bài tập chương	6 bài Ch1–6, nộp qua GitHub, chấm theo rubric	**0.15**
Kiểm tra giữa kỳ	Trắc nghiệm + đoạn code ngắn, 30 phút, tuần 8 (Ch1–3)	**0.1**
Thuyết trình nhóm	Tuần 15, chủ đề mở rộng	**0.1**
Đồ án EduHub Web	Nhóm ≤ 3: sản phẩm deploy + báo cáo + vấn đáp	**0.55**

Bảng 1.1. Trọng số các thành phần đánh giá.

1.1

PHẦN 2 / 8

PHẦN 2 · MỤC 1.1

Gõ URL rồi chuyện gì xảy ra?

- Client – server, URL, DNS
- HTTP/HTTPS, request, response, status code
- Trình duyệt dựng trang: HTML → DOM, CSS → CSSOM, render

1.1 Ví dụ đời thường: gọi món ở quán ăn

🙋 Khách gọi món

Bạn (**client**) ghi phiếu: "1 cơm tấm". Phiếu đó là **request**.

👨‍🍳 Bếp nấu

Bếp (**server**) nhận phiếu, chuẩn bị món theo đúng yêu cầu.

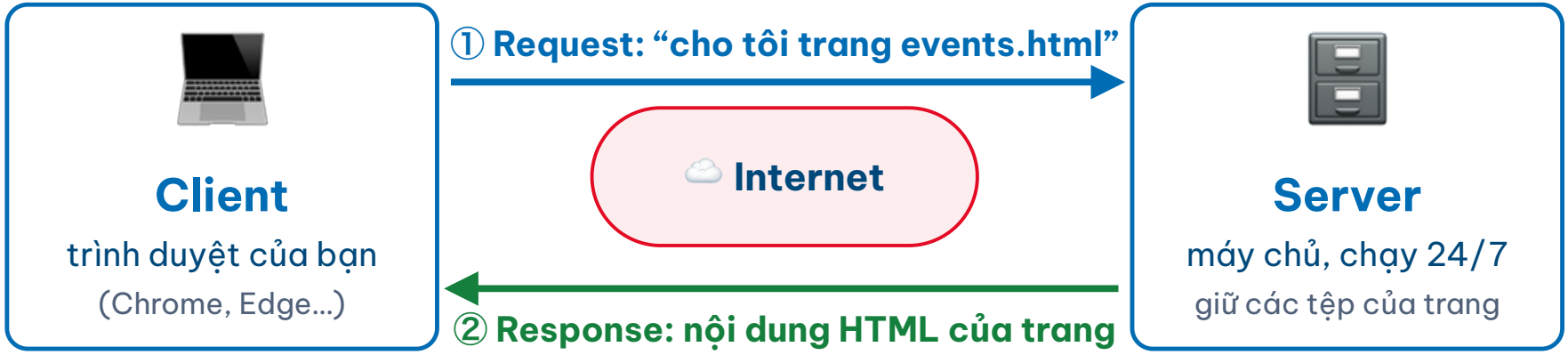
🍽️ Mang món ra

Món ăn mang ra là **response**. Hết món? Bếp báo "hết" — giống lỗi **404**.

📒 Sổ địa chỉ quán

Muốn đến quán phải biết địa chỉ — đó là việc của **DNS**.

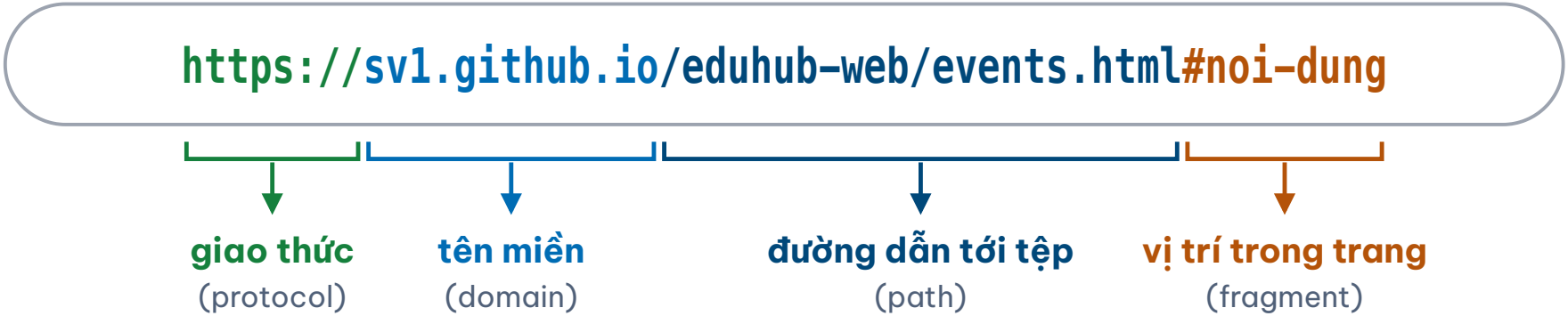
ACTIVE RECALL Trong quán ăn, ai là client, ai là server? Một quán có thể phục vụ bao nhiêu khách cùng lúc?



Hình 1.3. Mô hình client – server: request đi, response về (tự vẽ).

Client luôn hỏi trước, **server** chỉ trả lời. Frontend (môn này) = phần chạy ở client.

Một URL gồm những phần nào?

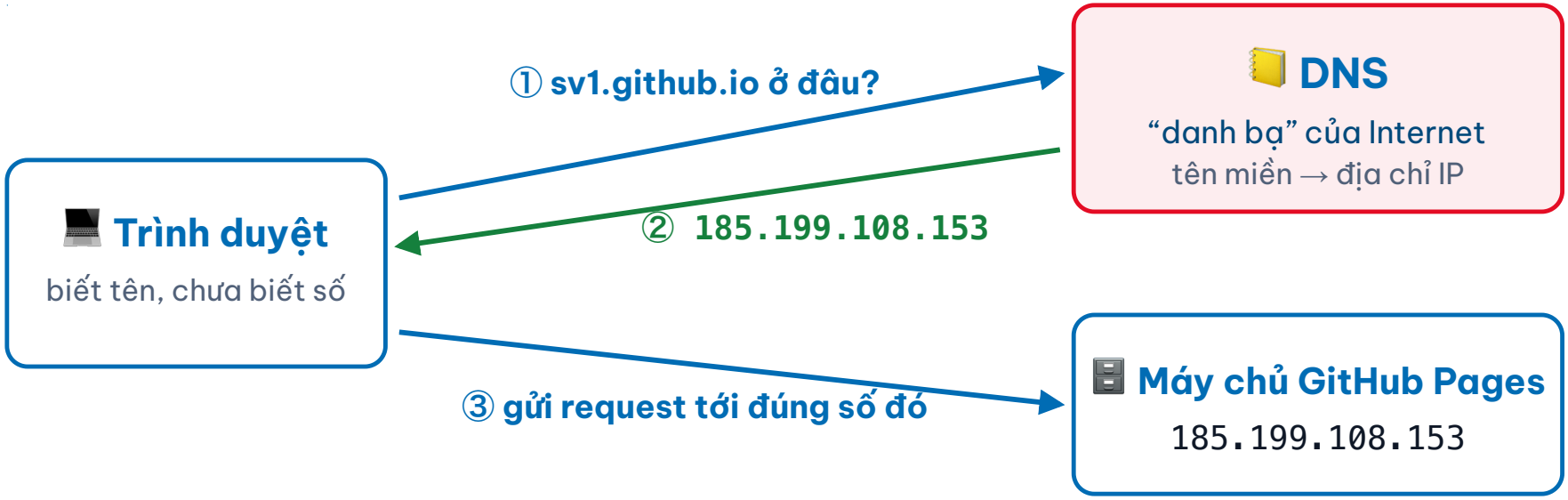


Hình 1.4. Các phần của một URL (tự vẽ).

URL (Uniform Resource Locator) = địa chỉ đầy đủ của *một* tài nguyên trên web: trang, ảnh, tệp...

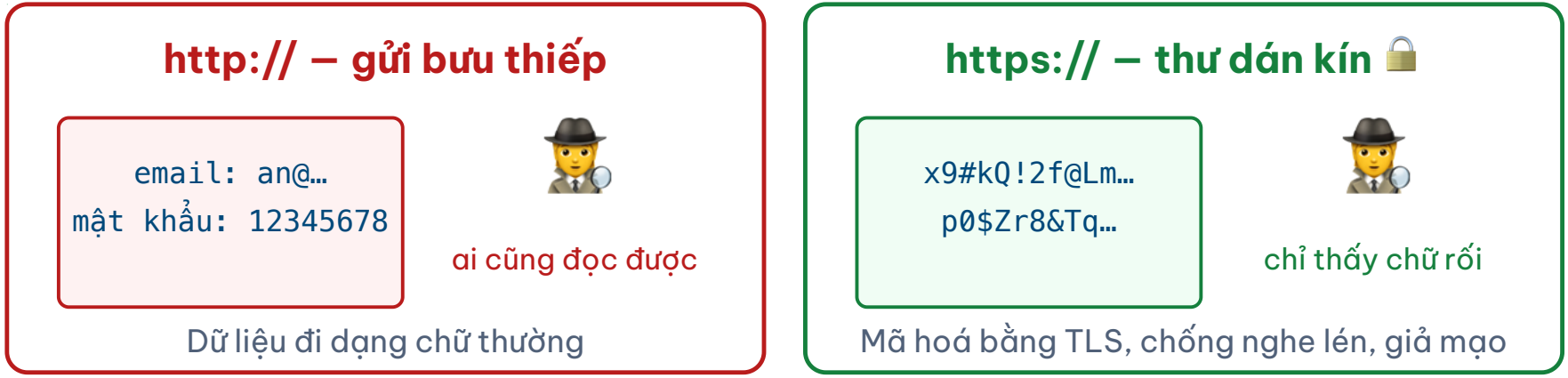
ACTIVE RECALL URL của trang chủ EduHub Web trên GitHub Pages của bạn sẽ là gì?

1.1 DNS – cuốn danh bạ của Internet



Hình 1.5. DNS đổi tên miền thành địa chỉ IP (tự vẽ).

Máy tính chỉ hiểu **địa chỉ IP** (dãy số). Con người nhớ **tên miền**. DNS (Domain Name System) tra tên → số, giống tra danh bạ tìm số điện thoại.



Hình 1.6. HTTP gửi chữ thường, HTTPS mã hoá đường truyền (tự vẽ).

HTTP = "ngôn ngữ" client và server nói với nhau. **HTTPS** = HTTP + mã hoá.
GitHub Pages bật HTTPS sẵn; trang có form đăng nhập *bắt buộc* phải HTTPS.

REQUEST (client → server)

```
GET /eduhub-web/events.html HTTP/1.1
Host: sv1.github.io
Accept: text/html
Accept-Language: vi
```

dòng 1: method + đường dẫn
các dòng sau: header (thông tin kèm)

RESPONSE (server → client)

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 5120

<!DOCTYPE html>
<html lang="vi">…
```

status · header · dòng trống · body (HTML)

Hình 1.7. Nội dung một request và một response HTTP (tự vẽ).

Minh hoạ dạng chữ; tab Network của DevTools hiện đúng các thông tin này.

1.1 Method và status code

Method = muốn làm gì

METHOD	Ý NGHĨA
`GET`	Lấy về (xem trang)
`POST`	Gửi lên (tạo mới)
`PUT`	Sửa
`DELETE`	Xoá

Bảng 1.2. Các method HTTP thường gặp.

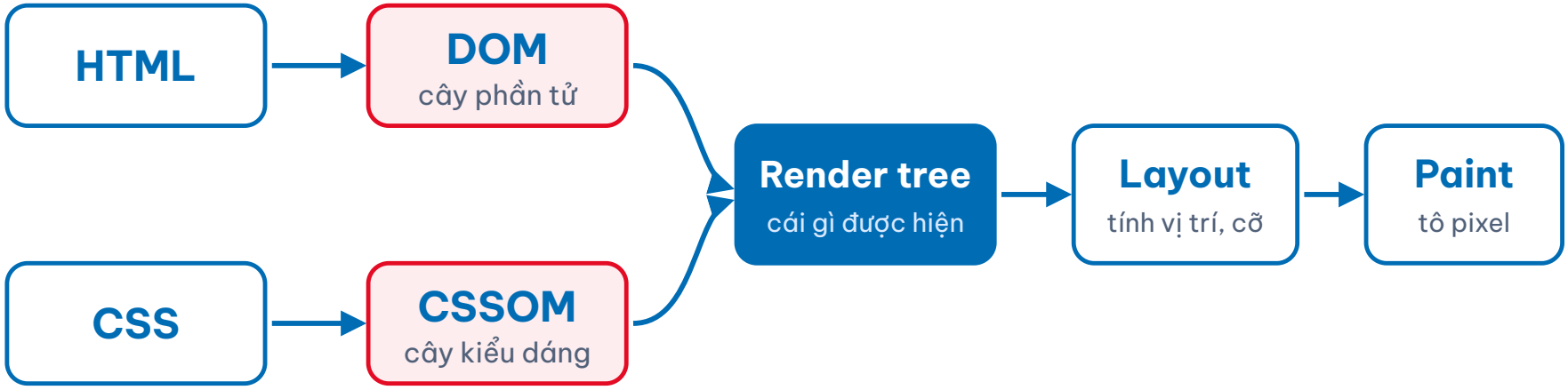
Ch1 chỉ dùng GET;
POST/PUT/DELETE học ở Ch3, Ch5.

Status code = kết quả ra sao

MÃ	NGHĨA	Ở QUÁN ĂN
`200`	OK	Có món, mang ra
`301`	Chuyển địa chỉ	Quán dời sang số mới
`304`	Dùng bản đã lưu	Món y như lần trước
`404`	Không tìm thấy	Thực đơn không có món
`500`	Lỗi server	Bếp bị cháy

Bảng 1.3. Các status code thường gặp.

2xx thành công · 3xx chuyển hướng ·
4xx lỗi phía client · 5xx lỗi phía server

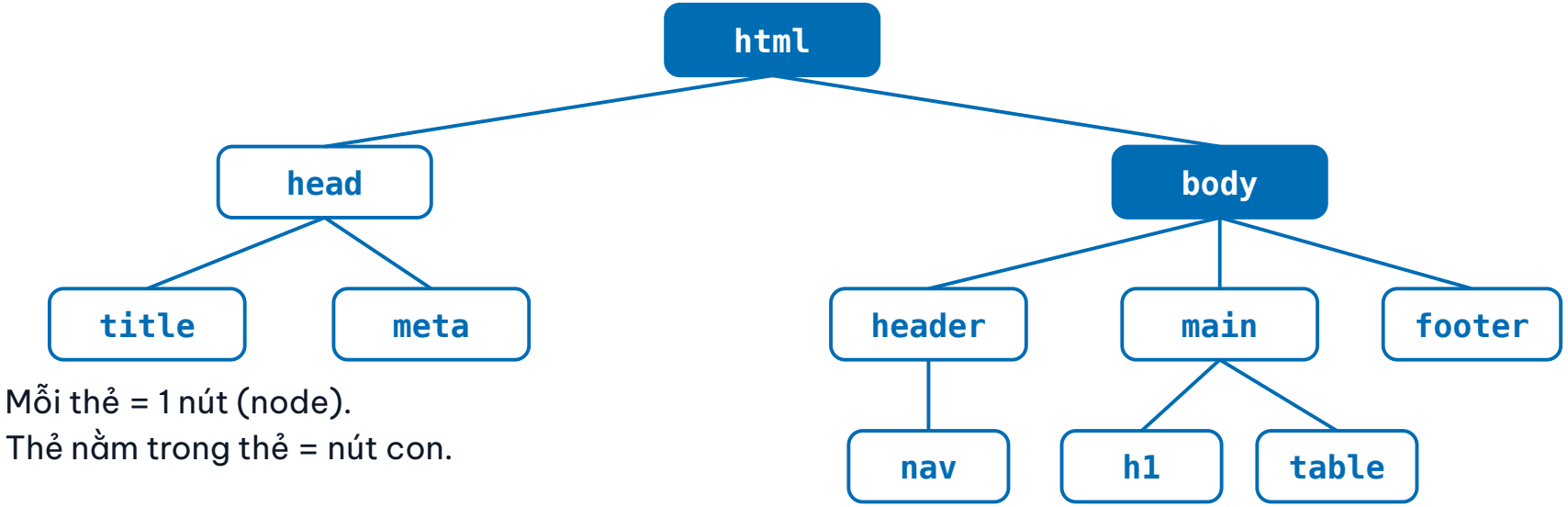


Hình 1.8. Các bước trình duyệt dựng trang (tự vẽ).

Như xây nhà: đọc **bản vẽ** (HTML → DOM), đọc **bảng màu sơn** (CSS → CSSOM), ghép lại, **đo đạc** vị trí (layout), rồi mới **sơn** (paint).

Ch1 chỉ có HTML (chưa có CSS riêng) nên trang hiện bằng kiểu mặc định của trình duyệt.

1.1 Cây DOM của trang events.html



Hình 1.9. Cây DOM rút gọn của `02_Projects/ch01/solution/events.html` (tự vẽ).

DOM (Document Object Model) = cây mà trình duyệt dựng từ HTML. Ch3 dùng JavaScript để sửa cây này; tab *Elements* của DevTools cho bạn xem nó ngay bây giờ.

1.1 Lỗi hay gặp — hiểu sai về web

✗ "Internet và Web là một"	✓ Internet = đường truyền; Web = các trang đi trên đường đó bằng HTTP
✗ "404 nghĩa là server hỏng"	✓ 404 = gõ sai/thiếu tệp (lỗi 4xx phía client). Server hỏng là 5xx
✗ "Trang có ổ khoá thì an toàn tuyệt đối"	✓ HTTPS chỉ mã hoá đường truyền; trang lừa đảo vẫn có thể có HTTPS
✗ Mở trang, sửa HTML, không thấy đổi	✓ Lưu tệp (Ctrl+S) rồi tải lại; trình duyệt có thể dùng bản cũ (304)

Thử ngay: soi một request bằng DevTools

Thử ngay · 3–5 phút

1. Mở `events.html` của EduHub (GV phát) bằng Live Server, bấm `F12` → tab **Network**.
2. Bấm `Ctrl+R` tải lại. Đếm số request. Status code của `events.html` là gì?
3. Bấm vào dòng `events.html` → xem *Headers*: method? `Content-Type`?
4. Sửa URL thành `…/khong-co.html`. Status code mới là gì?
5. Tab **Elements**: tìm thẻ `<table>` trong cây DOM.

Đáp án gợi ý — Thử ngay 1.1

- Vài request: tài liệu `events.html`, ảnh `assets/logo.svg` và đoạn script Live Server tự chèn để tự tải lại.
- `events.html`: **200** (hoặc **304** nếu trình duyệt đã có bản lưu).
- Method **GET**; `Content-Type: text/html; charset=UTF-8`.
- `khong-co.html` → **404 Not Found**.
- `<table>` nằm trong `body → main`.

HỎI THÊM Vì sao ảnh logo là một request riêng, không đi chung với HTML?

1.2

PHẦN 3 / 8

PHẦN 3 · MỤC 1.2

Frontend, Backend và Full-stack

- Ai làm phần nào trong một website
- Ba cách làm ra trang: static, SPA, SSR

1.2 Ví dụ đời thường: một nhà hàng

🪑 Phòng ăn = Frontend

Bàn ghế, thực đơn, nhân viên phục vụ — mọi thứ khách **nhìn thấy, chạm vào**.

🍳 Nhà bếp = Backend

Nấu nướng, tính tiền, quản lý kho — khách **không thấy** nhưng thiếu thì không có món.

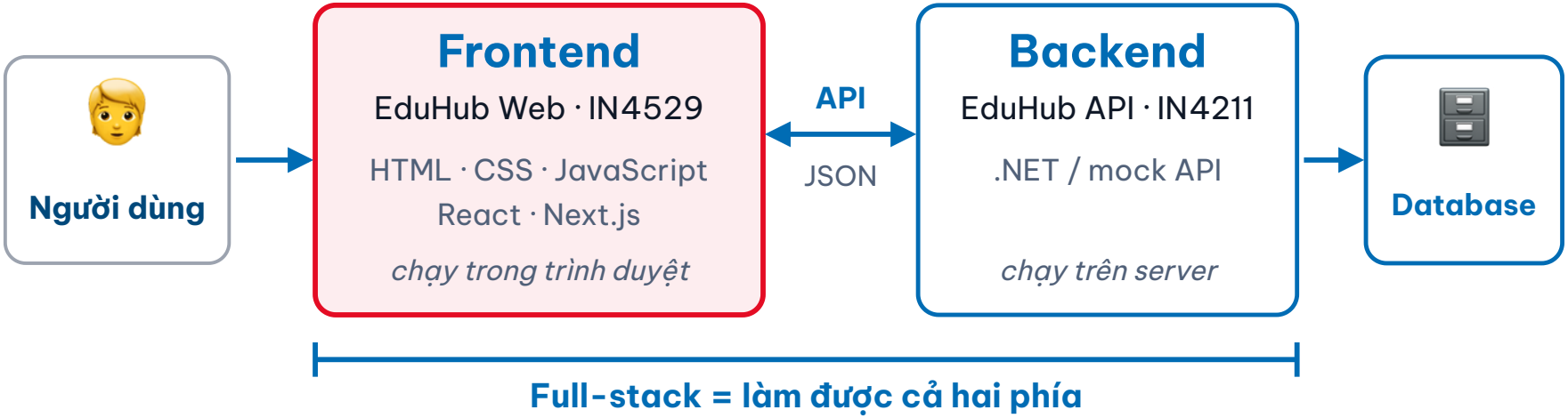
🥫 Kho = Database

Nơi cất nguyên liệu (dữ liệu) lâu dài.

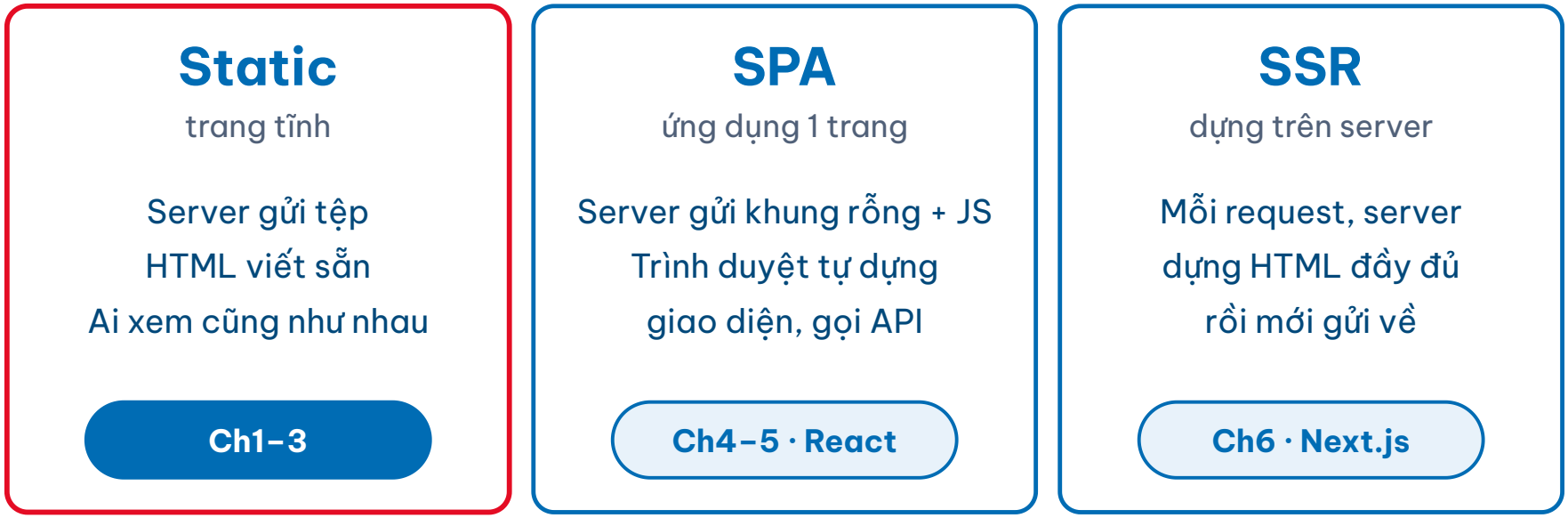
👨‍🍳 Chủ quán làm hết = Full-stack

Vừa đứng bếp vừa phục vụ: biết cả hai phía.

ACTIVE RECALL Nút "Đăng ký" trên trang thuộc phòng ăn hay nhà bếp? Còn việc kiểm tra MSSV đã tồn tại chưa?



Hình 1.10. Frontend, backend và database trong EduHub (tự vẽ).



Hình 1.11. Ba cách làm ra trang: static, SPA, SSR (tự vẽ).

Hôm nay chỉ cần nhớ **tên** và **ai dựng HTML**. Học kỹ ở Ch4 (SPA) và Ch6 (SSR).

1.2 Lỗi hay gặp – FE/BE

✗ "Frontend chỉ là vẽ cho đẹp"	✓ Frontend còn lo dữ liệu, trạng thái, tốc độ, accessibility
✗ Kiểm tra dữ liệu ở frontend là đủ	✓ Frontend kiểm để báo lỗi nhanh; backend **vẫn phải** kiểm lại
✗ "SPA lúc nào cũng tốt hơn trang tĩnh"	✓ Chọn theo nhu cầu: trang giới thiệu → static; app nhiều tương tác → SPA
✗ Để mật khẩu, khoá bí mật trong HTML/JS	✓ Ai cũng xem được mã frontend (View source) – bí mật để ở backend

Thử ngay · 3 phút

Mỗi việc dưới đây thuộc **Frontend** hay **Backend**?

1. Hiện bảng lịch 8 sự kiện cho đẹp, dễ đọc
2. Lưu thông tin sinh viên vừa đăng ký vào cơ sở dữ liệu
3. Báo "MSSV phải có đúng 10 chữ số" ngay khi bấm Đăng ký
4. Kiểm tra email đã có người dùng chưa
5. Mở/đóng phần trả lời trong mục "Câu hỏi thường gặp"

Đáp án gợi ý — Thử ngay 1.2

1. **Frontend** — hiển thị (HTML bảng, CSS ở Ch2).
2. **Backend** — chỉ server mới ghi được vào database.
3. **Frontend** (`pattern="[0-9]{10}"`) — và backend *kiểm lại*.
4. **Backend** — chỉ server biết danh sách email (trả 409 nếu trùng).
5. **Frontend** — thẻ `<details>`, không cần server.

1.3

PHẦN 4 / 8

PHẦN 4 · MỤC 1.3

Môi trường làm việc

- Trình duyệt và DevTools: Elements, Network, Console
- VS Code và extension Live Server
- Node.js 24 LTS và terminal cơ bản

1.3 Ví dụ đời thường: hộp đồ nghề của lập trình viên

Chrome + DevTools
= kính lúp và máy đo
Soi cấu trúc, request, lỗi (phím F12)

VS Code
= bàn làm việc
Viết code, có gợi ý, báo lỗi

Live Server
= cửa sổ xem thử
Lưu tệp là trang tự tải lại

Terminal
= ra lệnh bằng chữ
Gõ lệnh thay vì bấm chuột

Node.js 24 LTS
= động cơ chạy công cụ
Chạy công cụ kiểm tra HTML

Git + GitHub
= điểm lưu game, tủ trên mây
Lưu lịch sử code, nộp bài

Hình 1.12. Hộp đồ nghề của Chương 1 (biểu tượng Chrome, VS Code, Node.js, Git: Devicon, giấy phép MIT; còn lại: tự vẽ).

Hướng dẫn cài từng bước nằm trong `README.md` của starter Ch1 — làm trước buổi thực hành đầu tiên.

DevTools: 3 tab dùng nhiều nhất



Hình 1.13. Ba tab DevTools dùng nhiều nhất (tự vẽ).

Mở bằng `F12` hoặc chuột phải → *Inspect*.

Sửa trong Elements chỉ là tạm — tải lại là mất.

Cài đặt

- Tải ở `code.visualstudio.com`
- Mở thư mục: *File* → *Open Folder…* (mở cả thư mục, không mở lẻ tệp)
- Mở terminal: *Terminal* → *New Terminal*
- Gõ `!` rồi Tab trong tệp `.html` → khung HTML5 sẵn

Extension cho Ch1

- **Live Server** (Ritwick Dey): chuột phải `index.html` → *Open with Live Server*, lưu là tự tải lại
- Tab Extensions: `Ctrl+Shift+X` (macOS `Cmd+Shift+X`)
- Trang mở ở địa chỉ `127.0.0.1:5500`

Theo `02_Projects/ch01/starter/README.md` — Bước 1 và Bước 3

```
pwd                 # đang đứng ở thư mục nào?
ls                  # liệt kê tệp (Windows PowerShell cũng dùng được)
cd eduhub-web       # đi vào thư mục eduhub-web
cd ..               # lùi ra thư mục cha
mkdir assets        # tạo thư mục mới
code .              # mở thư mục hiện tại bằng VS Code

node -v             # kiểm tra Node.js, mong đợi v24.x
git --version       # kiểm tra Git
```

Đoạn mã 1.1. Các lệnh terminal cơ bản.

Terminal luôn chạy lệnh **trong thư mục hiện tại** — đứng sai chỗ là lệnh làm sai chỗ.

Node.js chạy JavaScript ngoài trình duyệt. **LTS** = bản hỗ trợ lâu dài, ổn định. `npx` = tải và chạy một công cụ mà không cần cài hẳn.

```
# Cần Node.js 24 LTS (html-validate mới yêu cầu Node >= 22.22)
npx -y html-validate@latest "*.html"     # kỳ vọng: không in lỗi, exit 0
```

Đoạn mã 1.2. `02_Projects/ch01/solution/README.md`

```
events.html
  52:5  error  <td> element is not permitted as content under <table>
✖ 1 problem (1 error, 0 warnings)
```

Ví dụ output khi có lỗi (minh hoạ) — đọc số dòng:cột để tìm chỗ sai.

1.3 Lỗi hay gặp — môi trường

✗ Bấm đúp mở `file:///…/index.html`	✓ Mở bằng Live Server (`http://127.0.0.1:5500`) như trang thật
✗ `'node' is not recognized…` ngay sau khi cài	✓ Đóng và mở lại terminal / VS Code để nhận PATH mới
✗ Chạy lệnh ở sai thư mục	✓ `pwd` trước, rồi `cd` vào đúng thư mục dự án
✗ Sửa ở tab Elements rồi tưởng đã lưu	✓ DevTools chỉ sửa tạm; sửa thật trong VS Code rồi Ctrl+S

Thử ngay: kiểm tra đồ nghề

Thử ngay · 3–5 phút

1. Mở terminal, chạy `node -v` và `git --version`. Chụp kết quả.
2. Tạo thư mục `thu-html`, mở bằng `code .`, tạo `index.html`, gõ `!` + Tab.
3. Đổi `<title>` thành tên bạn, thêm `<h1>Xin chào</h1>` trong `body`.
4. Open with Live Server. Sửa chữ trong `h1`, Ctrl+S — trang tự đổi?
5. F12 → Console: có dòng đỏ nào không?

Gợi ý: `node -v` in `v24.…`; nếu báo "not recognized" → mở lại terminal. Không có dòng đỏ là đạt.

1.4

PHẦN 5 / 8

PHẦN 5 · MỤC 1.4

Git và GitHub cơ bản

- init, add, commit, push
- Commit message rõ nghĩa
- Đưa trang lên mạng bằng GitHub Pages

1.4 Ví dụ đời thường: lưu game theo từng màn

💾 Commit = điểm lưu game

Qua mỗi màn lưu một lần. Chơi hỏng → quay lại điểm lưu.

🧺 Staging = giỏ đồ trước khi tính tiền

Chọn những tệp nào sẽ đi vào lần lưu tới (`git add`).

☁️ GitHub = tủ cất trên mây

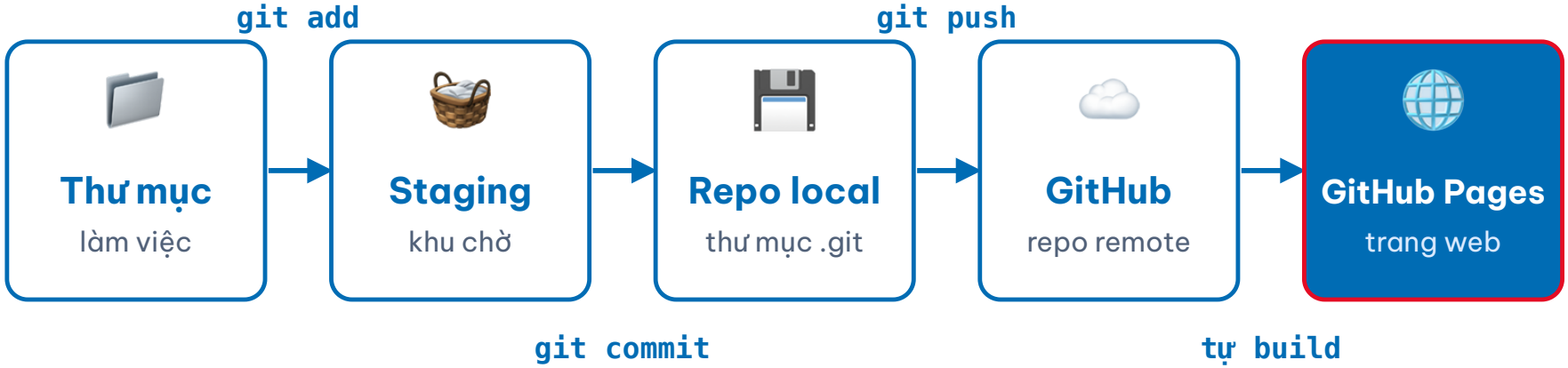
Máy hỏng vẫn còn code; GV xem được bài; nhóm cùng làm.

🌐 GitHub Pages = treo bài lên bảng tin

Biến repo thành trang web ai cũng mở được.

ACTIVE RECALL Bạn đã từng có thư mục `baitap_final`, `baitap_final2`, `baitap_final_that_su`? Git giải quyết đúng chuyện đó.

1.4 Git workflow: từ máy bạn lên Internet



Hình 1.14. Luồng làm việc Git: từ máy bạn lên GitHub Pages (tự vẽ).

```
git config --global user.name "Nguyen Van A"          # làm 1 lần trên máy
git config --global user.email "mssv@sv.dthu.edu.vn"

git init                                               # biến thư mục thành repo
git add .                                              # đưa mọi thay đổi vào staging
git commit -m "Khởi tạo EduHub Web từ starter Ch01"   # tạo điểm lưu
git branch -M main
git remote add origin https://github.com/<ten-tai-khoan>/EduHubWeb-Ch01-<MSSV>.git
git push -u origin main                                # đẩy lên GitHub
```

Đoạn mã 1.3. `02_Projects/ch01/starter/README.md — Bước 1, Bước 2`

Từ lần sau chỉ cần: `git add .` → `git commit -m "…"` → `git push`. Xem trạng thái bất cứ lúc nào: `git status`.

1.4 Commit message thế nào là tốt?

✗ Khó hiểu

```
git commit -m "update"
git commit -m "sua"
git commit -m "asdfgh"
```

Đoạn mã 1.4. Commit message khó hiểu.

Một tuần sau không ai biết lần lưu đó làm gì.

✓ Rõ nghĩa

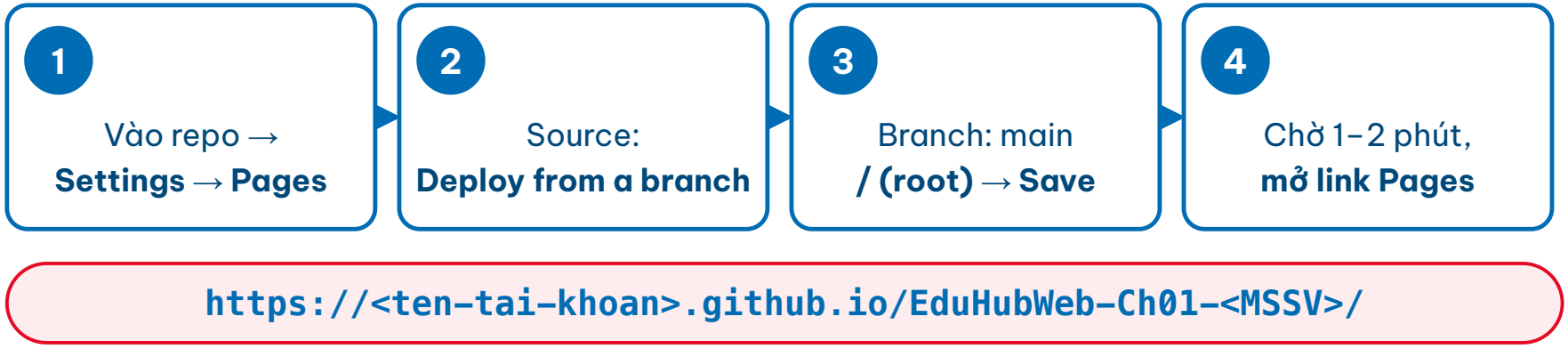
```
git add .
git commit -m "Thêm trang lịch sự kiện (ch1-T2)"
git push
```

Đoạn mã 1.5. Commit message rõ nghĩa — theo `02_Projects/ch01/starter/README.md`, Bước 4.

Nói **đã làm gì**, kèm mã nhiệm vụ.

Quy tắc Ch1: **xong mỗi nhiệm vụ commit một lần**; bài nộp cần ≥ 3 commit rõ nghĩa.

1.4 Bật GitHub Pages



Hình 1.15. Các bước bật GitHub Pages (tự vẽ).

Mỗi lần `git push`, trang trên mạng tự cập nhật sau khoảng một phút. Dán link vào `README.md` để nộp.

Theo `02_Projects/ch01/starter/README.md` — Bước 5

1.4 Lỗi hay gặp — Git & Pages

✗ Đặt tên `Events.html`, link ghi `events.html`	✓ Pages phân biệt hoa/thường: đặt tên tệp toàn chữ thường
✗ `git commit` báo "nothing added to commit"	✓ Quên `git add .` trước khi commit
✗ Trang chủ trên Pages báo 404	✓ Phải có `index.html` ở gốc repo, chọn đúng nhánh `main`
✗ Tick "Add a README" khi tạo repo rồi push bị từ chối	✓ Tạo repo trống; starter đã có README sẵn

Thử ngay · 5 phút

1. Tạo repo **public** tên `thu-html` trên GitHub (không tick README).
2. Trong thư mục `thu-html` ở phần 1.3: `git init` → `git add .` → `git commit -m "Trang đầu tiên"`.
3. Chạy 3 lệnh `branch -M`, `remote add`, `push` theo trang GitHub hướng dẫn.
4. Bật Pages, gửi link vào nhóm lớp. Ai mở được trang của bạn bên cạnh?

Gợi ý: kiểm tra trên GitHub thấy `index.html` ở gốc repo; link dạng `https://ten-tai-khoan.github.io/thu-html/`.

1.5

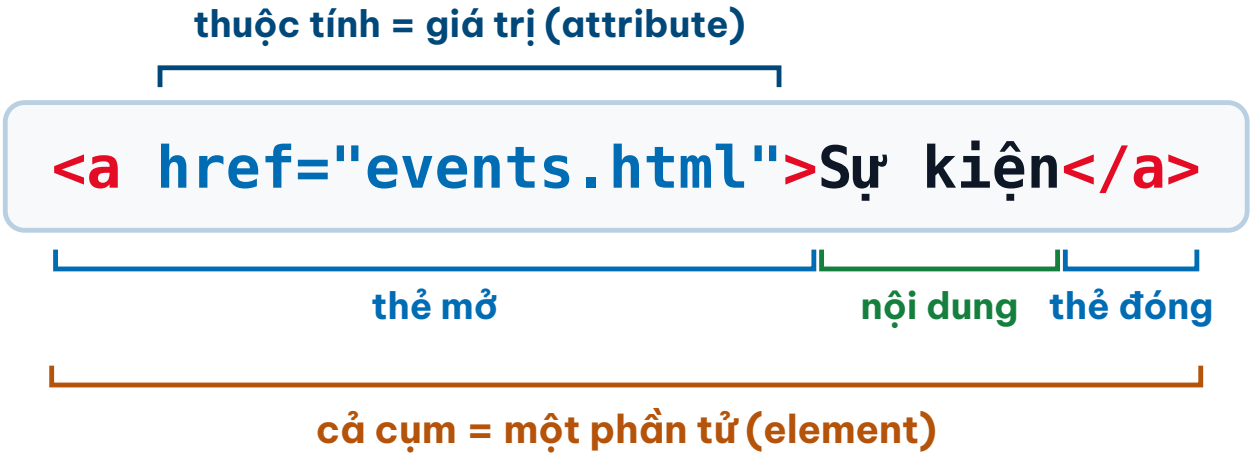
PHẦN 6 / 8

PHẦN 6 · MỤC 1.5

HTML5

- Khung tài liệu và thẻ semantic
- Liên kết, ảnh, danh sách, bảng
- Form, input types, validation gốc
- `<details>` và mở rộng `<dialog>`

1.5 Ví dụ đời thường: thẻ HTML như nhãn dán lên hộp



Hình 1.16. Cấu tạo một phần tử HTML (tự vẽ).

HTML không "vẽ" gì cả — nó **dán nhãn** cho nội dung: đây là tiêu đề, đây là liên kết, đây là bảng. Trình duyệt và máy đọc màn hình dựa vào nhãn để hiểu.

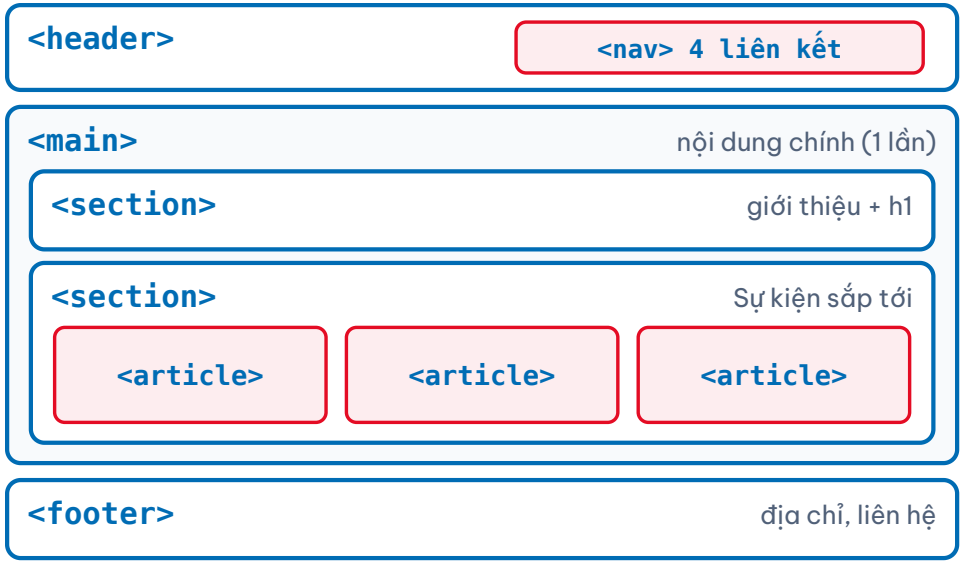
Khung một tài liệu HTML5

```html
<!DOCTYPE html>
<html lang="vi">
  <head>
    <meta charset="utf-8">
    <title>Lịch sự kiện · EduHub Web</title>
  </head>
  <body>
    <main id="noi-dung">
      <h1>Lịch sự kiện</h1>
    </main>
  </body>
</html>
```

Đoạn mã 1.6. `02_Projects/ch01/solution/events.html (rút gọn: bỏ meta viewport — học ở Ch2)`

`head` = thông tin *về* trang · `body` = thứ người dùng thấy · `lang="vi"`: đọc đúng tiếng Việt · `charset`: không lỗi dấu.

1.5 Thẻ semantic: chia trang thành vùng có nghĩa



Hình 1.17. Bố cục semantic của trang chủ EduHub (tự vẽ).

- `header` – đầu trang
- `nav` – khối điều hướng
- `main` – nội dung chính, mỗi trang **một**
- `section` – một phần có tiêu đề
- `article` – mẩu đứng riêng được (1 sự kiện, 1 bài viết)
- `footer` – chân trang

header + nav trong EduHub

```html
<a href="#noi-dung">Bỏ qua, đến nội dung chính</a>
<header>
  <a href="index.html">
    <img src="assets/logo.svg" alt="EduHub – về trang chủ" width="48" height="48">
  </a>
  <nav aria-label="Điều hướng chính">
    <ul>
      <li><a href="index.html" aria-current="page">Trang chủ</a></li>
      <li><a href="events.html">Sự kiện</a></li>
      <li><a href="index.html#cau-lac-bo">Câu lạc bộ</a></li>
      <li><a href="register.html">Đăng ký thành viên</a></li>
    </ul>
  </nav>
</header>
```

Đoạn mã 1.7. `02_Projects/ch01/solution/index.html`

Menu là **danh sách liên kết** → `nav > ul > li > a`.

main · section · article

```html
<main id="noi-dung">
  <section aria-labelledby="su-kien-sap-toi">
    <h2 id="su-kien-sap-toi">Sự kiện sắp tới</h2>

    <article>
      <h3><a href="event-detail.html">Workshop: Làm website đầu tiên với HTML</a></h3>
      <img src="assets/club-tin-hoc.svg" alt="Biểu tượng CLB Tin học"
           width="640" height="360" loading="lazy">
      <p>
        <time datetime="2026-10-17T07:30+07:00">07:30, thứ Bảy 17/10/2026</time> ·
        Phòng máy B1.203 · CLB Tin học
      </p>
    </article>
    …
  </section>
</main>
```

Đoạn mã 1.8. `02_Projects/ch01/solution/index.html (rút gọn)`

“Div soup” và HTML semantic

✗ Toàn div

```
<div class="top">
  <div class="menu">…</div>
</div>
<div class="content">
  <div class="title">Sự kiện</div>
  <div class="item">…</div>
</div>
<div class="bottom">…</div>
```

Đoạn mã 1.9. “Div soup”: chỉ dùng `div`.

Máy đọc màn hình, Google không biết đâu là menu, đâu là tiêu đề.

✓ Semantic

```
<header>
  <nav>…</nav>
</header>
<main>
  <h1>Sự kiện</h1>
  <article>…</article>
</main>
<footer>…</footer>
```

Đoạn mã 1.10. Cùng nội dung viết bằng thẻ semantic.

Cùng hiển thị như nhau, nhưng **có nghĩa**: dễ đọc, dễ tìm, dễ style.

1.5 Liên kết `<a>`: đi đâu cũng được

```html
<a href="events.html">Sự kiện</a>                    <!-- tệp cùng thư mục -->
<a href="index.html#cau-lac-bo">Câu lạc bộ</a>       <!-- tới id trong trang khác -->
<a href="#noi-dung">Bỏ qua, đến nội dung chính</a>   <!-- tới id trong trang này -->
<a href="https://github.com/signup">github.com</a>   <!-- trang web khác -->
<a href="mailto:eduhub@dthu.edu.vn">eduhub@dthu.edu.vn</a>  <!-- mở ứng dụng email -->
```

Đoạn mã 1.11. `02_Projects/ch01/solution/index.html, event-detail.html`

KIỂU ĐƯỜNG DẪN	VÍ DỤ	KHI NÀO DÙNG
Tương đối	`events.html`	Trang trong cùng website — chạy được cả local lẫn Pages
Tuyệt đối	`https://…`	Website khác
Mảnh `#id`	`#noi-dung`	Nhảy tới phần tử có `id` đó

Bảng 1.4. Ba kiểu đường dẫn trong `href`.

1.5 Ảnh `<img>`: alt, kích thước, lazy

```
<img src="assets/hero.svg"
     alt="Minh hoạ ba sinh viên đứng cạnh bảng CLB và bảng Sự kiện"
     width="1200" height="500">

<img src="assets/club-tin-hoc.svg" alt="Biểu tượng CLB Tin học"
     width="640" height="360" loading="lazy">
```

Đoạn mã 1.12. `02_Projects/ch01/solution/index.html`

- `alt` — chữ thay thế: máy đọc màn hình đọc to, hiện ra khi ảnh lỗi. Tả **nội dung**, không ghi "ảnh 1"
- `width` / `height` — trình duyệt giữ chỗ trước, trang không bị "giật"
- `loading="lazy"` — ảnh ở dưới màn hình đầu chỉ tải khi sắp cuộn tới. Ảnh đầu trang thì **không** lazy

1.5 Danh sách: `ul`, `ol`, `dl`

```
<ol>
  <li>Trình duyệt hiển thị một
      trang web như thế nào.</li>
  <li>Viết trang HTML đầu tiên
      trong VS Code.</li>
</ol>
<ul>
  <li>Máy tính xách tay …</li>
  <li>Một tài khoản GitHub …</li>
</ul>
```

```
<dl>
  <dt>Thời gian</dt>
  <dd><time datetime=
   "2026-10-17T07:30+07:00">07:30
   – 09:30, thứ Bảy
   17/10/2026</time></dd>
  <dt>Số chỗ</dt>
  <dd>60 (còn 12 chỗ)</dd>
</dl>
```

Đoạn mã 1.13. `02_Projects/ch01/solution/event-detail.html` (rút gọn, xuống dòng cho vừa slide).

`ol` = có thứ tự (các bước) · `ul` = không thứ tự · `dl` = cặp "tên – giá trị" · `<time datetime>` = thời gian máy đọc được (ISO).



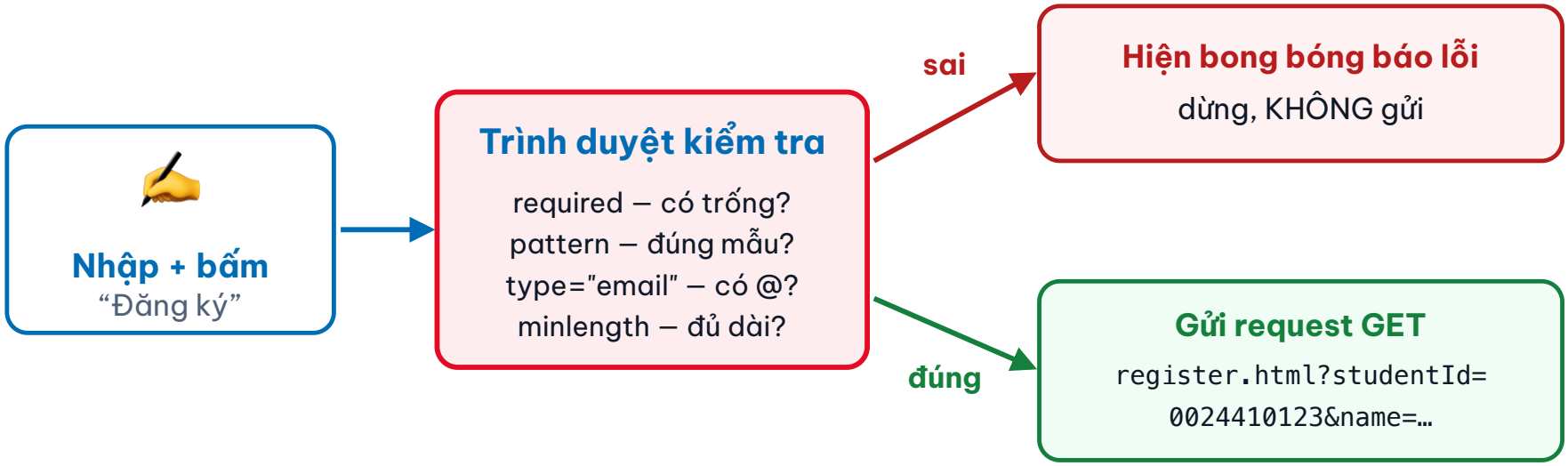
Hình 1.18. Các phần của bảng lịch sự kiện (tự vẽ).

Bảng dùng cho **dữ liệu dạng lưới** (lịch, điểm, giá). **Không** dùng bảng để dàn trang — việc đó là của CSS (Ch2).

Bảng lịch sự kiện — code

```
<table>
  <caption>Sự kiện sắp tới của các câu lạc bộ (giờ Việt Nam)</caption>
  <thead>
    <tr>
      <th scope="col">Thời gian</th>
      <th scope="col">Sự kiện</th> …
    </tr>
  </thead>
  <tbody>
    <tr>
      <td><time datetime="2026-10-17T07:30+07:00">07:30, T7 17/10/2026</time></td>
      <th scope="row">
        <a href="event-detail.html">Workshop: Làm website đầu tiên với HTML</a>
      </th>
      …
```

Đoạn mã 1.14. `02_Projects/ch01/solution/events.html (rút gọn, xuống dòng cho vừa slide)`



Hình 1.19. Luồng gửi form có validation gốc (tự vẽ).

Validation gốc = kiểm tra bằng thuộc tính HTML, chưa cần JavaScript. Ch1 chưa có server nên form gửi `GET` về chính trang để thấy dữ liệu trên thanh địa chỉ; Ch3 gửi tới API.

Form đăng ký (1): label, fieldset, pattern

```html
<form action="register.html" method="get">
  <fieldset>
    <legend>Thông tin sinh viên</legend>
    <p>
      <label for="student-id">Mã số sinh viên <abbr title="bắt buộc">*</abbr></label>
      <input id="student-id" name="studentId" type="text" inputmode="numeric"
             required pattern="[0-9]{10}" maxlength="10"
             aria-describedby="student-id-hint" autocomplete="off">
      <small id="student-id-hint">Gồm đúng 10 chữ số, ví dụ 0024410123.</small>
    </p>
    …
  </fieldset>
```

Đoạn mã 1.15. `02_Projects/ch01/solution/register.html`

`for` của label = `id` của input → bấm chữ là con trỏ nhảy vào ô. `name` = tên trường khi gửi đi.

Form đăng ký (2): email, mật khẩu, chọn

```
<input id="email" name="email" type="email" required autocomplete="email"
       placeholder="mssv@sv.dthu.edu.vn">
<input id="password" name="password" type="password" required minlength="8"
       autocomplete="new-password" aria-describedby="password-hint">

<select id="club" name="club">
  <option value="">— Chưa chọn —</option>
  <option value="Tin học">CLB Tin học</option>
  …
</select>

<input id="year-1" name="year" type="radio" value="1" checked>
<label for="year-1">Năm 1</label>
<input id="agree" name="agree" type="checkbox" required>
```

Đoạn mã 1.16. `02_Projects/ch01/solution/register.html (rút gọn)`

1.5 Input types và thuộc tính kiểm tra

TYPE	DÙNG CHO
`text`	Chữ thường
`email`	Email (phải có @)
`password`	Mật khẩu (che chữ)
`date`	Chọn ngày
`radio`	Chọn 1 trong nhóm (cùng `name`)
`checkbox`	Đánh dấu có/không

Bảng 1.5. Các input type thường dùng.

THUỘC TÍNH	Ý NGHĨA
`required`	Không được bỏ trống
`pattern`	Phải khớp mẫu (regex)
`minlength`	Ít nhất N ký tự
`maxlength`	Nhiều nhất N ký tự
`min` / `max`	Giới hạn số, ngày
`autocomplete`	Gợi ý trình duyệt tự điền

Bảng 1.6. Các thuộc tính kiểm tra dữ liệu.

Còn `select` (danh sách chọn), `textarea` (đoạn văn nhiều dòng), `button type="submit|reset"`. Dữ liệu từ form vẫn phải kiểm lại ở backend.

1.5 `<details>`: mở/đóng không cần JavaScript

```html
<section aria-labelledby="hoi-dap">
  <h2 id="hoi-dap">Câu hỏi thường gặp</h2>
  <details>
    <summary>Ai được đăng ký thành viên EduHub?</summary>
    <p>Tất cả sinh viên đang học tại Trường Đại học Đồng Tháp,
       chỉ cần mã số sinh viên và email.</p>
  </details>
  …
</section>
```

Đoạn mã 1.17. `02_Projects/ch01/solution/index.html (rút gọn, xuống dòng cho vừa slide)`

▶ **Bấm thử: Một sinh viên được tham gia mấy câu lạc bộ?**

`summary` = dòng luôn hiện; bấm vào thì phần còn lại mở ra. Hợp cho FAQ.

MỞ RỘNG `<dialog>`: hộp thoại gốc của HTML

Hộp thoại xác nhận (“Bạn chắc chắn?”) có sẵn trong HTML. Thuộc tính `open` cho nó hiện; nút trong `form method="dialog"` đóng hộp thoại.

```
<dialog id="xac-nhan" open>
  <p>Bạn chắc chắn muốn huỷ đăng ký sự kiện này?</p>
  <form method="dialog">
    <button value="huy">Huỷ đăng ký</button>
    <button value="giu">Giữ lại</button>
  </form>
</dialog>
```

Đoạn mã 1.18. Minh hoạ ngoài solution Ch1 (EduHub Ch1 không dùng `dialog`). Mở dạng modal bằng `showModal()` cần JavaScript — học ở Ch3.

1.5 Lỗi hay gặp — HTML

✗ Hai thẻ `h1`, hoặc nhảy h2 → h4 cho chữ nhỏ	✓ Một `h1`/trang; cấp heading theo cấu trúc, cỡ chữ để CSS lo
✗ `<img src="logo.svg">` không alt	✓ Luôn có `alt` mô tả + `width` / `height`
✗ `<label>Email</label>` `<input id="email">`	✓ `<label for="email">` khớp đúng `id`
✗ Dùng `<b>`, `<font>`, `<center>`, bảng để dàn trang	✓ Thẻ semantic cho nghĩa; giao diện để CSS (Ch2)
✗ Quên đóng thẻ, lồng sai `<p><div>`	✓ Chạy `npx -y html-validate@latest "*.html"`

Thử ngay: dựng thẻ một sự kiện

Thử ngay · 5 phút

Trong tệp `index.html` ở phần 1.3, viết một `<article>` cho sự kiện:

> "**Ngày hội Hiến máu tình nguyện** — 07:00, thứ Bảy 24/10/2026 · Hội trường lớn · CLB Tình nguyện"

1. Tiêu đề `h3` có liên kết tới `events.html`
2. Thời gian bọc trong `<time datetime="…">` đúng ISO
3. Thêm một ô nhập email `required` có `label` đúng chuẩn

Đáp án gợi ý — Thử ngay 1.5

```html
<article>
  <h3><a href="events.html">Ngày hội Hiến máu tình nguyện</a></h3>
  <p>
    <time datetime="2026-10-24T07:00+07:00">07:00, thứ Bảy 24/10/2026</time> ·
    Hội trường lớn · CLB Tình nguyện
  </p>
</article>
```

Đoạn mã 1.19. `02_Projects/ch01/solution/index.html`

```html
<label for="email">Email <abbr title="bắt buộc">*</abbr></label>
<input id="email" name="email" type="email" required autocomplete="email"
       placeholder="mssv@sv.dthu.edu.vn">
```

Đoạn mã 1.20. `02_Projects/ch01/solution/register.html`

1.6

PHẦN 7 / 8

PHẦN 7 · MỤC 1.6

Accessibility và SEO cơ bản

- Heading đúng thứ tự, label, landmark
- Meta và Open Graph
- Chấm điểm trang bằng Lighthouse

Ví dụ đời thường: lối đi cho mọi người

♿ Accessibility (a11y)

Như dốc cho xe lăn, chữ nổi trong thang máy: trang web **ai cũng dùng được** — người khiếm thị dùng máy đọc màn hình, người chỉ dùng bàn phím.

🔎 SEO

Như biển hiệu và địa chỉ rõ ràng: **Google, Zalo, Facebook** hiểu trang nói gì để hiện đúng khi tìm kiếm, chia sẻ.

Tin vui: HTML semantic làm đúng ở phần 1.5 là đã được **hơn nửa đường** cho cả hai.

ACTIVE RECALL Thử nhắm mắt, nghe máy đọc một trang toàn `div`: bạn có biết đâu là menu không?

1.6 Heading đúng thứ tự = mục lục của trang

✓ Dàn ý trang chủ (index.html)

- h1 EduHub Web — Cổng CLB & sự kiện sinh viên
  - h2 Sự kiện sắp tới
    - h3 Workshop: Làm website đầu tiên…
    - h3 Ngày hội Hiến máu tình nguyện
  - h2 Câu lạc bộ
    - h3 CLB Tin học · CLB Tình nguyện …
  - h2 Câu hỏi thường gặp

✗ Nhảy cấp

- h1 EduHub Web
  - h4 Sự kiện
  - h2 …
- h1 Câu lạc bộ

máy đọc màn hình "lạc"

Hình 1.20. Dàn ý heading: đúng và sai (tự vẽ).

Người dùng máy đọc màn hình nhảy theo heading như lật mục lục. Một `h1`; h1 → h2 → h3, không nhảy cấp.

Label, landmark, bỏ qua menu

```html
<a href="#noi-dung">Bỏ qua, đến nội dung chính</a>
<nav aria-label="Điều hướng chính"> … </nav>
<li><a href="events.html" aria-current="page">Sự kiện</a></li>

<main id="noi-dung">
  <section aria-labelledby="su-kien-sap-toi">
    <h2 id="su-kien-sap-toi">Sự kiện sắp tới</h2>
```

Đoạn mã 1.21. `02_Projects/ch01/solution/index.html, events.html`

- **Landmark** = vùng có tên: `header`, `nav`, `main`, `footer` — máy đọc nhảy thẳng tới được
- `aria-label` / `aria-labelledby` — đặt tên cho vùng; `aria-current="page"` — "bạn đang ở đây"
- Liên kết "Bỏ qua…" giúp người dùng bàn phím (phím Tab) khỏi đi qua cả menu

Meta và Open Graph

```
<title>Lịch sự kiện · EduHub Web</title>
<meta name="description" content="Lịch các sự kiện sắp tới của câu lạc bộ sinh viên
      Trường Đại học Đồng Tháp, học kỳ I năm học 2026–2027.">
<!-- Open Graph: thay "ten-tai-khoan" bằng tên tài khoản GitHub … -->
<meta property="og:title" content="Lịch sự kiện · EduHub Web">
<meta property="og:description" content="Lịch các sự kiện sắp tới của câu lạc bộ …">
<meta property="og:url"
      content="https://ten-tai-khoan.github.io/eduhub-web/events.html">
<meta property="og:image"
      content="https://ten-tai-khoan.github.io/eduhub-web/assets/og-image.png">
<link rel="icon" href="assets/logo.svg" type="image/svg+xml">
```

Đoạn mã 1.22. `02_Projects/ch01/solution/events.html (rút gọn, xuống dòng cho vừa slide)`

`title` + `description` → kết quả Google. `og:*` → thẻ xem trước khi dán link vào Zalo/Facebook. `og:url`, `og:image` phải là URL **tuyệt đối**.

1.6 Open Graph hiện ra thế nào khi chia sẻ?



Hình 1.21. Thẻ xem trước khi chia sẻ link (Open Graph) (tự vẽ; ảnh `og-image.png` lấy từ đáp án mẫu).

Kiểm tra: dán link GitHub Pages của bạn vào Zalo/Messenger. Không có ảnh? Xem lại `og:image` có đúng URL tuyệt đối và tệp đã push chưa.

1.6 Lighthouse: chấm điểm trang



Hình 1.22. Bốn nhóm điểm của Lighthouse (tự vẽ).

1. Mở trang trên GitHub Pages → `F12` → tab **Lighthouse**
2. Chọn chế độ **Mobile**, tick Accessibility + SEO → *Analyze page load*
3. Đọc từng mục đỏ/cam: thường là thiếu `alt`, thiếu `label`, thiếu `meta description`

Mục tiêu bài Challenge ch1-T6: Accessibility ≥ 95 và SEO ≥ 95.

1.6 Lỗi hay gặp — a11y & SEO

✗ Mọi trang cùng `<title>EduHub Web</title>`

→ ✓ Mỗi trang một title riêng: "Lịch sự kiện · EduHub Web"

✗ `alt="ảnh"`, `alt="image1.png"`

→ ✓ Tả điều ảnh muốn nói; ảnh trang trí thuần thì `alt=""`

✗ `og:image` = `assets/og-image.png`

→ ✓ URL tuyệt đối, bắt đầu bằng `https://`

✗ Chỉ có `placeholder`, không có `label`

→ ✓ Placeholder biến mất khi gõ; luôn có `label`

Thử ngay: chỉ dùng bàn phím

Thử ngay · 5 phút

1. Mở `register.html` của EduHub (GV phát). Cất chuột đi.
2. Bấm `Tab` liên tục: phần tử nào được chọn đầu tiên? Có đi hết form được không?
3. Bấm vào chữ "Mã số sinh viên" (khi có chuột):
   con trỏ có nhảy vào ô không? Vì sao?
4. Chạy Lighthouse (Mobile) cho trang. Accessibility được bao nhiêu điểm?

Gợi ý: Tab đầu tiên rơi vào "Bỏ qua, đến nội dung chính";
bấm chữ nhảy vào ô vì `label for="student-id"` khớp `id`.



PHẦN 8 / 8

PHẦN 8

Bài tập và tổng kết

- Bài tập Chương 1 — EduHub Web 4 trang
- Tổng kết và câu hỏi tự kiểm tra
- Chuẩn bị cho Chương 2

Bài tập Chương 1 — EduHub Web 4 trang

MÃ	CẤP	VIỆC	ĐIỂM
`ch1-T1`	Warm-up	Trang chủ `index.html`: semantic, 3 sự kiện, 6 CLB, FAQ `details`	20
`ch1-T2`	Applied	`events.html`: bảng 8 sự kiện, `caption`, `th scope`, `time`	20
`ch1-T3`	Applied	`event-detail.html`: `article`, `dl`, `ol`, `ul`	15
`ch1-T4`	Applied	`register.html`: form, label, validation gốc	25
`ch1-T5`	Applied	Meta/SEO, Open Graph; GitHub Pages; ≥ 3 commit	20
`ch1-T6`	Challenge	Liên kết "Bỏ qua…", Lighthouse a11y + SEO ≥ 95	+10

Bảng 1.7. Các nhiệm vụ của bài tập Chương 1.

Starter: `02_Projects/ch01/starter/` — tìm các ghi chú `TODO(ch1-Tk)`, chữ có sẵn trong `content/noi-dung-mau.md`. Nộp: link repo + link GitHub Pages.

Kết quả cần đạt: EduHub Web sau Chương 1



index.html
Trang chủ · ch1-T1



events.html
Lịch sự kiện · ch1-T2



event-detail.html
Chi tiết · ch1-T3



register.html
Đăng ký · ch1-T4

Hình 1.23. Bốn trang EduHub Web của đáp án mẫu, chưa có CSS (ảnh chụp `02_Projects/ch01/solution/`).

LƯU Ý Trang trông "thô" là **đúng**: Chương 1 chỉ có HTML, chưa có CSS. Chương 2 sẽ làm đẹp — không viết lại HTML.

Tổng kết Chương 1

- Gõ URL → **DNS** tra IP → **request** HTTP(S) → server trả **response** + status code → trình duyệt dựng **DOM**, render
- **Frontend** chạy ở trình duyệt, **backend** ở server; static · SPA · SSR khác nhau ở *ai dựng HTML*
- Đồ nghề: DevTools (Elements/Network/Console), VS Code + Live Server, Node.js 24 LTS, terminal
- Git: `init → add → commit → push`; GitHub Pages đưa trang lên mạng
- HTML = **dán nhãn có nghĩa**: semantic, `alt`, bảng có `caption/th scope`, form có `label` + validation gốc
- Heading đúng cấp, landmark, meta, Open Graph → ai cũng dùng được, Google/Zalo hiểu được

Câu hỏi tự kiểm tra

✔ CHECKPOINT — CHƯƠNG 1

1. Kể theo thứ tự các bước từ lúc gõ URL đến lúc thấy trang. DNS làm gì?
2. Status code 404 và 500 khác nhau thế nào? Lỗi nào do phía client?
3. Vì sao nên dùng `<main>`, `<nav>` thay vì `<div>`? Kể 2 lợi ích.
4. Viết một ô nhập MSSV bắt buộc, đúng 10 chữ số, có label.
5. `git add` và `git commit` khác nhau ở đâu?

NHÌN LẠI Quay lại 3 câu "Bạn đã biết gì?" đầu buổi — giờ bạn trả lời khác đi thế nào?

Chuẩn bị cho Chương 2 — CSS hiện đại và Responsive

Chương 2 học gì?

- Selector, cascade, specificity
- Box model, đơn vị `rem`, `clamp()`
- Flexbox và Grid để dàn trang
- Responsive mobile-first, dark mode
- Design token bằng CSS custom properties

4 trang HTML của bạn sẽ được **làm đẹp** — không viết lại HTML.

Cần chuẩn bị

- Hoàn thành ch1-T1…T5, push lên GitHub, bật Pages
- Chạy `npx -y html-validate@latest "*.html"` → 0 lỗi
- Cài Node.js 24 LTS nếu chưa có
- Đọc MDN "CSS first steps" (developer.mozilla.org)
- Mở DevTools, thử đổi màu chữ một trang bất kỳ trong tab Elements → Styles